Included Use Cases

Table of Contents

- Introduction
- MPD Use Cases
  - UTC Timing related use cases
  - Labels-related use cases
  - Track ordering related use cases
  - Accessibility setting for subtitles
  - Content tailoring to regions or devices
  - Adaptation Sets Switching
  - Adding a "value" attribute to EventStreams
- HLS Use Cases
  - Default Subtitle/Audio Language
  - Accessibility settings for HLS Live in absence of encoder support
  - Creating main playlist variants when statically packaging HLS

Introduction

The previous chapters have shown how *Manifest Edit* comes with a preinstalled Plugins Library and how these can be combined, by means of the Pipeline Configuration File, into pipelines.

We encourage users to think creatively about the possible different ways of combining plugins to realize specific functionalities they may need. However, we think it's very useful to provide a list of specific use cases that Unified Streaming has encountered and how they can be solved by *Manifest Edit*.

MPD Use Cases

UTC Timing related use cases

Live DASH manifests indicate a default `time.akamai.com` iso time server.

Possible related issue:

- the default time server is unreachable by any one of the downstream elements

- I cannot use my own time server or a third-party other
- I cannot switch to a different time format (i.e. direct, iso, ...)
- I cannot add a list of time servers for resiliency

Consequences:

- playback does not start at the live edge
- AV playback drifting issues

Solution: use the descriptor_remove plugin and descriptor_add plugin to change the UTCTiming element in the MPD root.

In this example, the customer wanted to use its own implementation of a time server, using the NTP protocol and related time format. At this purpose, both the format and value of the UTCTiming element needs to be changed. This can be accomplished by first removing the original element, then adding one in the desired format.

```
mpd:
- manifest_edit.plugins.mpd.descriptor_remove:
    plugin_config:
      name: 'UTCTiming'
      schemeIdUri: '.*'

- manifest_edit.plugins.mpd.descriptor_add:
    plugin_config:
      name: 'UTCTiming'
      schemeIdUri: 'urn:mpeg:dash:utc:ntp:2014'
      value: 'https://myserver.mydomain/?ntp'
```

You can find a similar configuration in the provided `utc_change.yaml` file available in the example folder.

Labels-related use cases

Possible related issue:

- a player/device User Interface must show track descriptions based on track properties (i.e. language) or group them by some kind of category
- the logic to generate User Interface track description is included in the player/device code

Consequences:

- any change in the User Interface messages involves a deployment of a new player/firmware

- poor user experience with low quality track descriptions

Solution: use the label add plugin and label remove plugin to add/edit labels that any front end software can use and show in their User Interface.

In this example, the customer wanted to show a description of the track's language in their player (i.e. "English" for a "lang=en" track). The player used Labels to do so but not all manifest tracks had Labels available.

Using the Label Add plugin, Labels can be added where absent (subtitle tracks in this case) and the language name can be obtained by making use of the `ISO639Name` utility function:

```
mpd:
  - manifest_edit.plugins.mpd.label_add:
      periods:
        - id: ".*"
          adaptationSets:
          - contentType :  "text"
            plugin_config:
              label:
                lang: "{lang}"
                data: "{ISO639Name(lang)} subtitles"
```

Track ordering related use cases

Based on the DASH standard, the order in which representations and Adaptation sets appear in a manifest is not meaningful.

Possible related issue:

- devices with poor standard implementations care for the order in which tracks appear or make erroneous assumptions about it
- UIs reflect the order in which audio, video or subtitle tracks appear in the manifest instead of the desired one

Consequences:

- playback starts with the wrong audio
- playback always starts with the lowest or the highest bitrate video tracks
- audio or subtitle track selection User Interfaces show out-of-order lists

Solution: use manifest_order plugin to reorder tracks in the order in which you wish them to appear in a manifest.

These example use cases are all realized applying different configurations to the *manifest_order* plugin.

You can find other examples of similar configurations in the provided `adaptation_sets_order.yaml`, `adaptation_sets_representations_order.yaml`, `representations_order.yaml` files available in the example folder.

Set First Track

In this use case, a single Representation is selected from a "video" adaptation set and moved so it appears before all others. Specifically, this use case requires a track with a specific language to be the first.

```
mpd:
- manifest_edit.plugins.mpd.manifest_order:
    periods:
      - '*' : '.*'
        adaptationSets:
          plugin_config:
            manualOrder:
              - lang: 'en'
                contentType: 'video'
```

Order Tracks by language

In this use case, all adaptation sets are sorted by language, in ascending alphabetical order.

```
mpd:
- manifest_edit.plugins.mpd.manifest_order:
    periods:
      - '*' : '.*'
        adaptationSets:
          plugin_config:
            orderBy :      "lang"
            orderCriteria: "asc"
```

Order Tracks in order of descending bitrate

In this use case, all adaptation sets are sorted by bandwidth, in descending order.

```
mpd:
- manifest_edit.plugins.mpd.manifest_order:
    periods:
      - '*' : '.*'
        adaptationSets:
          plugin_config:
            orderBy :      "bandwidth"
            orderCriteria: "desc"
```

Accessibility setting for subtitles

Possible related issue:

- best practices and regulations may impose the presence of "hard of hearing" subtitle tracks
- vod content or encoder settings may lack to signal the tracks appropriately, which in turn affects DASH signaling

Consequences:

- subtitle track will not be rendered
- "hard of hearing" subtitle may not be present in the streaming

Solution:: explicitly set a "text" track of your choice with the correct "hard of hearing" signaling, using the descriptor_add plugin for DASH and the default_language plugin for HLS.

For DASH, this means appropriately settings the Accessibility and Role elements in those Adaptation Sets having a contentType set to *text*, which is achievable by means of a combination of the descriptor_add and descriptor_remove plugins. The manifest in fact could already include Accessibility and Role elements, probably with different schemeIdUri and value content.

It is best to first remove the existing Accessibility and Role and then add the ones that are surely compatible with the device.

The resulting configuration is:

```
mpd:
  - manifest_edit.plugins.mpd.descriptor_remove:
      periods:
        - '*' : '.*'
          adaptationSets:
            - contentType :  "text"
              plugin_config:
                name: "Accessibility"
            - contentType :  "text"
              plugin_config:
                name: "Role"

  - manifest_edit.plugins.mpd.descriptor_add:
      periods:
        - '*' : '.*'
          adaptationSets:
            - contentType :  "text"
              plugin_config:
                name: "Accessibility"
                schemeIdUri: "urn:tva:metadata:cs:AudioPurposeCS:2007"
                value: "2"
            - contentType :  "text"
              plugin_config:
                name: "Role"
                schemeIdUri: "urn:mpeg:dash:role:2011"
                value: "main"
```

For HLS, this means setting the CHARACTERISTICS attribute in the EXT-X-MEDIA tag of a the desired track in a main HLS playlist. This is achievable using the default_language plugin, using a configuration similar to the following, which sets the french track as "hard of hearing":

```
m3u8_main:
- manifest_edit.plugins.m3u8_main.default_language:
   - type: 'SUBTITLES'
     language: 'fr'
     characteristics: "public.accessibility.describes-spoken-
dialog,public.accessibility.describes-music-and-sound"
     # autoselect will be added implicitly as required by HLS authoring guidelines
```

Content tailoring to regions or devices

By default a manifest returns all streams available in the server manifest.

Possible related issue:

- some audio, video or subtitle tracks are only necessary in certain geographical regions
- regulations or rights management may require to limit some content to only specific reasons or devices

- some devices are not compatible with one or more codecs used in the stream
- it is desirable to stream HD-only or SD-only to certain devices or regions

Consequences:

- rights infringement
- poor user experience
- device or player crashes

Solution: create variants of a given manifest, tailored to specific regions or devices, by using element_remove plugin

Depending on your requirement, you may want to remove entire Adaptation Sets or specific Representations.

This configuration removes all HD video Representations (e.g. width>=720) from a DASH manifest, leaving only SD tracks (also adjusting the 'maxWidth', 'minWidth', 'maxBitrate', 'minBitrate' attributes of the corresponding Adaptation Set). Such a configuration may be suitable to provide SD-only content to specific customer tiers or to simplify the manifest for non-HD-capable devices or players:

```
mpd:
- manifest_edit.plugins.mpd.element_remove:
    periods:
      - '*' : '.*'
        adaptationSets:
          - contentType: "video"
            representations:
              # matches width >= 720
              - width: '^([8-9]\d+|[7][2-9]\d|\d{4,})$'
                plugin_config:
                  remove: 'this'
```

When providing multiple video or audio encoding, it may be useful to remove one or more to force specific devices and players to use only the other codecs.

I.e. this example removes all audio tracks with a 'dtse' codec:

```
mpd:
- manifest_edit.plugins.mpd.element_remove:
    periods:
      - '*' : '.*'
        adaptationSets:
          - codecs : 'dtse'
            plugin_config:
              remove: 'this'
```

I.e. this example removes all video tracks with an 'hevc' codec:

```
mpd:
- manifest_edit.plugins.mpd.element_remove:
    periods:
      - '*' : '.*'
        adaptationSets:
          - codecs : '^hevc.*'
            plugin_config:
              remove: 'this'
```

Refer to the element_remove plugin doc page for a complete overview on the selection options.

Adaptation Sets Switching

Sometimes, similar tracks are split in different Adaptation Sets (i.e. when using different DRM keys).

Possible related issue:

- some players and devices may not be able to switch across tracks belonging to different Adaptation Sets
- some audio, video or subtitle tracks are not played back

Consequences:

- poor user experience
- broken playback
- device or player crashes

Solution: enable adaptation set switching using adaptation_sets_switching plugin (requires a supported player).

In this use case, the customer wants to offer the same content with different encryption keys for different resolutions (HD and SD). The DASH implementation guidelines (Chapter 7) clearly states that in this case, two different adaptation sets must be created. This is the default behavior of *Unified Origin*.

For such scenarios, the separate adaptation sets can be added to a same Switching Set to enable seamless switching functionality in players supporting it.

The following example adds all the "video" adaptation sets of a manifest to the same Switching Set:

```
mpd:
- manifest_edit.plugins.mpd.adaptation_sets_switching:
    periods:
      - '*' : '.*'
        adaptationSets:
          - contentType: 'video'
            plugin_config:
              switching_set: '1'
              destination: 'supplemental_property'

- manifest_edit.plugins.mpd.descriptor_add:
```

You can find a similar configuration in the provided `adaptation_sets_switching.yaml` file available in the example folder.

Adding a "value" attribute to EventStreams

This is an example of specific device bugs that can be worked around using *Manifest Edit*

Possible related issue:

- a specific set-top-box crashes if a "value" field is not present in the EventStream element

Consequences:

- broken playback for a specific audience
- need to wait for a firmware update from the set-top-box manufacturer

Solution: enable a workaround using eventstream_value_add plugin.

Despite recognizing that "value" is an optional attribute and that this behaviour is clearly faulty on the device side, this plugin seems the easiest way forward to provide compatibility in a short time.

The specific content of the "value" element is not important, however this plugin allows to configure it. We will just use the string `"1"` in this example.

This is the resulting configuration:

```
mpd:
- manifest_edit.plugins.mpd.eventstream_value_add:
    value: "1"
```

HLS Use Cases

Default Subtitle/Audio Language

When multiple audio/subtitle languages are present, different regions should playback by default different languages.

Possible related issue:

- playback starts with a foreign language
- the wrong subtitle track is enabled by default
- at every playback, users need to manually select audio/subtitles

Consequences:

- poor user experience

Solution: use the default_language plugin to choose default languages.

The customer needs to serve several regions with different languages. They need players to automatically start playback of the audio track with the region's main language. Unfortunately the DEFAULT audio track is determined at packaging time and that implies having different manifest per region, which is undesirable.

With *Manifest Edit*, it is possible to change the manifest on the fly and have audio or subtitle tracks default to a specific language.

The solution is based on having one pipeline configuration file per language and the use query parameters to select the pipeline applying the desired one.

For audio tracks (but the same applies to the subtitle case, just use `type: 'SUBTITLES'`), if we suppose to support two languages, i.e. Swedish and Danish, the solution would involve creating two pipeline configuration files such as the following:

default_swedish.yaml:

```
m3u8_main:
  - manifest_edit.plugins.m3u8_main.default_language:
    - type: 'AUDIO'
      language: 'sv'
      default: 'YES'
```

default_danish.yaml:

```
m3u8_main:
  - manifest_edit.plugins.m3u8_main.default_language:
    - type: 'AUDIO'
      language: 'da'
      default: 'YES'
```

If your configuration reflects the one suggested in the installation manual (see Manifest Edit-related configuration), then you would use either one of the following URL to select the desired language:

```
http://my-origin.com/my-video.ism/.mpd?python_pipeline_config=default_swedish
http://my-origin.com/my-video.ism/.mpd?python_pipeline_config=default_danish
```

Accessibility settings for HLS Live in absence of encoder support

Possible related issue:

- best practices and regulations may impose the presence of a "video description" audio track.
- live encoders are supposed to signal the accessibility properties of a track by adding a "kind" mp4 box to the track. Some commonly used encoders may not support such functionality.

Consequences:

- video description track does not have the correct accessibility settings and is indistinguishable from the main audio track.
- the video description track is not present in the streaming.

Solution:: explicitly set the "characteristics" property of your track. Often the main audio track will have the same TYPE, GROUP-ID and LANGUAGE than the accessible track, but it will have a URI that overrides the default one.

It is possible to configure the default language plugin to select and edit only the tracks whose URI field matches a specific pattern. Assuming you want to signal an audio track with "describes-video" characteristics, and assuming this track's URI contains the *_accessible* pattern, the resulting configuration would be:

```
m3u8_main:
- manifest_edit.plugins.m3u8_main.default_language:
   - type: 'AUDIO'
     characteristics: "public.accessibility.describes-video"
     # autoselect will be added implicitly as required by HLS authoring guidelines
```

Creating main playlist variants when statically packaging HLS

Possible related issue:

- when using HLS static packaging (--store-hls option), the generated main playlist includes all possible variants.

Consequences:

- this unique playlist may expose some clients to variants they may not be authorized/compatible with, causing security/stability issues.

Solution:: explicitly create versions of the main playlist that all share a common set of media playlists but only include a specific subset of variants, using the ext_x_media_remove plugin to choose default languages.

An example of such an use case could be the main playlist that is generated when statically packaging content with both a Dolby and an aac audio track. This is how such a playlist may look like:

```
#EXTM3U
#EXT-X-VERSION:6
## Created with Unified Streaming Platform  (version=1.12.9-28924)

# AUDIO groups
#EXT-X-MEDIA:TYPE=AUDIO,GROUP-ID="audio-aacl-
128",LANGUAGE="en",NAME="English",DEFAULT=YES,AUTOSELECT=YES,CHANNELS="2",URI="manifest-aac_dolby-
audio_eng=128000.m3u8"
#EXT-X-MEDIA:TYPE=AUDIO,GROUP-ID="audio-ec-3-
192",LANGUAGE="en",NAME="English",DEFAULT=YES,AUTOSELECT=YES,CHANNELS="6",URI="manifest-aac_dolby-
audio_eng_1=192000.m3u8"

# variants
#EXT-X-STREAM-INF:BANDWIDTH=401000,CODECS="mp4a.40.2,hev1.1.6.L93.B0",RESOLUTION=640x360,FRAME-
RATE=50,VIDEO-RANGE=SDR,AUDIO="audio-aacl-128",CLOSED-CAPTIONS=NONE
manifest-aac_dolby-video=250000.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=984000,CODECS="mp4a.40.2,hev1.1.6.L93.B0",RESOLUTION=960x540,FRAME-
RATE=50,VIDEO-RANGE=SDR,AUDIO="audio-aacl-128",CLOSED-CAPTIONS=NONE
manifest-aac_dolby-video=800000.m3u8

#EXT-X-STREAM-INF:BANDWIDTH=469000,CODECS="ec-3,hev1.1.6.L93.B0",RESOLUTION=640x360,FRAME-
RATE=50,VIDEO-RANGE=SDR,AUDIO="audio-ec-3-192",CLOSED-CAPTIONS=NONE
manifest-aac_dolby-video=250000.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1052000,CODECS="ec-3,hev1.1.6.L93.B0",RESOLUTION=960x540,FRAME-
RATE=50,VIDEO-RANGE=SDR,AUDIO="audio-ec-3-192",CLOSED-CAPTIONS=NONE
manifest-aac_dolby-video=800000.m3u8
```

Players receiving this playlist will be aware of both alternate Dolby and aac audio tracks.

It is possible to create a variant playlist that only includes an audio tracks subset (e.g. only the "aac" one) by means of the following configuration, which removes the Dolby track

```yaml
m3u8_main:
- manifest_edit.plugins.m3u8_main.ext_x_media_remove:
   - type: 'AUDIO'
     group-id: '.*-ec-3-.*'
```

Since this implies the complete removal of the "audio-ec-3-192" audio group, all variants referencing it are also completely removed, leading to the following resulting playlist

```
#EXTM3U
#EXT-X-VERSION:6
## Created with Unified Streaming Platform  (version=1.12.9-28924)

# AUDIO groups
#EXT-X-MEDIA:TYPE=AUDIO,GROUP-ID="audio-aacl-
128",LANGUAGE="en",NAME="English",DEFAULT=YES,AUTOSELECT=YES,CHANNELS="2",URI="manifest-aac_dolby-
audio_eng=128000.m3u8"

# variants
#EXT-X-STREAM-INF:BANDWIDTH=401000,CODECS="mp4a.40.2,hev1.1.6.L93.B0",RESOLUTION=640x360,FRAME-
RATE=50,VIDEO-RANGE=SDR,AUDIO="audio-aacl-128",CLOSED-CAPTIONS=NONE
manifest-aac_dolby-video=250000.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=984000,CODECS="mp4a.40.2,hev1.1.6.L93.B0",RESOLUTION=960x540,FRAME-
RATE=50,VIDEO-RANGE=SDR,AUDIO="audio-aacl-128",CLOSED-CAPTIONS=NONE
manifest-aac_dolby-video=800000.m3u8
```

By running Manifest Edit with different configurations it is possible to obtain other variants of Main Playlist that can be made available to players from the same folder where HLS have been packaged.